

System Design Interview Field Guide

A decision-first cheatsheet for Staff+ product and infrastructure interviews

THE STAFF+ DIFFERENCE

Strong candidates can draw a scalable system. Staff+ candidates also decide what **not** to build, connect technical choices to product semantics, quantify risk, expose failure modes, and describe a credible migration path.

THE META-PATTERN

For every box, state: (1) the constraint that forced it, (2) the failure mode it introduces, and (3) the simpler fallback at one-tenth the scale.

HOW TO USE THIS GUIDE

- §02–03 to rehearse the interview control loop.
- §04–09 as pattern lookup during focused practice.
- §10–12 for production, migration, and AI-native depth.
- §13 as a final five-minute drill before an interview.

§	SECTION	WHAT YOU NEED TO RETRIEVE
02	Run the room	45-minute loop, scope, rubric, decision log
03	Estimate and set SLOs	workload math, latency, availability, RPO/RTO
04	Shape APIs and data	contracts, schemas, indexes, pagination, IDs
05	Scale reads and writes	cache, replicas, partitioning, hot keys
06	Go asynchronous	queues, streams, delivery, outbox, backpressure
07	Protect correctness	races, transactions, consistency, coordination
08	Serve realtime and search	fanout, presence, reconnect, search, geo
09	Design for failure	timeouts, retries, shedding, multi-region, DR
10	Operate and evolve	observability, security, privacy, migrations
11	AI-native systems	streaming, RAG, agents, injection, evals
12	The LLM product blueprint NEW	ChatGPT-like walk-through, LLM estimation anchors, cost levers
13	Decision matrices and drill	store/transport choices, closing checklist

Numbers are estimation anchors, not capacity promises. Real throughput depends on payloads, access patterns, indexes, durability, hardware, and tail-latency targets.

02 Run the room: the Staff+ control loop

A. THE 45-MINUTE SHAPE

- **0-5 | Frame.** Confirm users, top 3-4 capabilities, out-of-scope items, and the one product invariant that must never break.
- **5-10 | Quantify.** Estimate read/write QPS, payload and storage growth, fanout, p95/p99 latency, availability, durability, and geography.
- **10-15 | Contract.** Define core entities, APIs/events, state transitions, and ownership boundaries before drawing infrastructure.
- **15-28 | Baseline.** Build the simplest end-to-end path that satisfies functional requirements. Walk one read and one write.
- **28-40 | Deep dive.** Attack the top two risks: scale, correctness, latency, failure, cost, security, or migration.
- **40-45 | Close.** Bottlenecks, degraded mode, observability, rollout, v1 cuts, and the next migration seam.

OPENING LINE

"I will first lock scope and the user-visible correctness rules, estimate the two dominant load paths, then build a simple baseline and spend the remaining time on the risks our numbers expose."

B. REQUIREMENTS THAT CHANGE ARCHITECTURE

- Ask only questions whose answers **change a decision**. Convert adjectives into measurable targets.
- **Scale:** DAU/MAU, actions per user, read:write ratio, peak multiplier, object sizes, retention, fanout distribution.
- **Latency:** p95/p99 for the user journey, not just average service latency. Separate first-byte/first-token from completion.
- **Correctness:** duplicates, ordering scope, stale reads, lost updates, exactly which state is authoritative.
- **Reliability:** SLO, data-loss tolerance, RPO/RTO, regional failure, offline/reconnect behavior.
- **Product:** who sees an update, how soon, edit/delete semantics, abuse controls, privacy and residency.

C. THE DECISION LOG

- Keep a small running table in the corner. It makes the conversation auditable and prevents circular design.

DECISION	WHY	COST / REVISIT SIGNAL
Cache-aside feed	100:1 reads; 30 s stale OK	Invalidation; <80% hit rate
Single-region primary	Writes need one order	RTO too high; region growth
Queue thumbnails	User need not wait	Lag; queue age exceeds SLO

D. STAFF+ SIGNALS VS ANTI-SIGNALS

SIGNAL	ANTI-SIGNAL
Makes a decision after naming tradeoffs	"It depends" without choosing
Prioritizes two risky paths deeply	Uniform shallow tour of every box
Connects failure to user impact	Lists replicas with no failure story
Names ownership and migration	Assumes a greenfield forever
Pushes back with quantified scope	Adds Kafka/cache/shards by reflex

E. WALK THE SYSTEM, DO NOT JUST DRAW IT

- **Write path:** auth → validation → idempotency → authoritative commit → event publication → projections → client confirmation.
- **Read path:** routing → authorization → cache/index → source of truth → pagination → freshness annotation.
- **Failure path:** dependency timeout → retry budget → degraded response or queue → alert → recovery/replay.
- At each hop say: latency, consistency, failure, ownership, and the metric that proves it works.

RECOVERY MOVE

If the interviewer redirects you: summarize your current decision in one sentence, accept the new constraint, and state which component or invariant changes. Steering is usually signal, not interruption.

03 Estimate and set SLOs

A. A FOUR-LINE CAPACITY MODEL

- Use powers of ten. Show the equation. Round visibly. Precision without measurements is false confidence.

```
avg_qps = active_users × actions_per_user / 86,400
peak_qps = avg_qps × peak_factor // test 2x, 5x, 10x
storage/day = writes_per_day × bytes_per_item × replication
concurrency = arrival_rate × time_in_system // Little's Law
```

EXAMPLE

100M DAU × 10 feed reads/day ÷ ~100k seconds/day ≈ 10k average QPS. At 5× peak and 2 KB responses, test around 50k QPS and 100 MB/s before cache amplification.

B. SIZE ANCHORS

- UTF-8 characters are 1–4 bytes; do not assume one character equals one byte.
- 64-bit integer: 8 B. UUID: 16 B raw, commonly 36 chars as text. Add row/index overhead.
- 1M × 1 KB logical records ≈ 1 GB before indexes, versions, replication, backups, and compression.
- One day = 86,400 s, close enough to 100k for interview math. One year ≈ 31.5M s.
- Always separate **logical data, working set, and physical footprint**.

C. LATENCY ORDER OF MAGNITUDE

OPERATION	ROUGH ORDER	DESIGN CONSEQUENCE
CPU/cache/RAM	ns to low μs	usually not the network design bottleneck
Local SSD	100 μs to few ms	access pattern and queueing dominate
Same-region RPC/cache	sub-ms to few ms	fanout multiplies tail latency
Indexed DB query	few to tens of ms	measure query shape and contention
Cross-region RTT	tens to 100+ ms	writes across regions pay physics
Model response	100 ms to many s	stream; cancel; set a time budget

- These are scale-of-magnitude anchors only. Queueing, payload, durability, region, hardware, and p99 targets can move them dramatically.

D. SLI → SLO → ERROR BUDGET

- **SLI**: measured user outcome, such as successful checkout under 500 ms.
- **SLO**: target, such as 99.9% of eligible checkouts in 30 days.
- **Error budget**: 1 – SLO. It funds controlled risk; 100% is usually economically wrong.
- Define the eligible population: exclude invalid requests; decide whether dependency failures count.
- Availability should be **end-to-end**: a 200 response with the wrong cart is not success.

E. AVAILABILITY, RPO, RTO

TERM	QUESTION	EXAMPLE
Availability	How often does the journey work?	99.9% monthly
RPO	How much committed data may be lost?	≤ 1 minute
RTO	How long until service is restored?	≤ 30 minutes
Durability	Will accepted data survive?	no lost paid order

SAY THIS

"I will not size nodes from a memorized vendor QPS. I will derive workload and bandwidth, choose an initial topology, then benchmark the exact payload and query at the target p99 with headroom."

F. DISTRIBUTIONS THAT CHANGE THE ANSWER

- Average hides burstiness. Ask about diurnal peaks, launches, retries, synchronized jobs, and regional skew.
- Power-law popularity creates celebrity keys and fanout outliers. Design for **p99 account size**, not only average followers.
- Tail latency compounds: a request that fans out to 50 dependencies is likely to encounter at least one slow call.
- Capacity target: expected peak + failure headroom + growth runway. State which one you are budgeting.

04 Shape APIs and data around access patterns

A. CONTRACT BEFORE COMPONENTS

- Define core resources, identifiers, ownership, state machine, and invariants.
- Show 2-4 representative APIs/events, not an exhaustive REST catalog.
- **Mutations:** validation, authZ, idempotency, concurrency token, response semantics.
- **Reads:** filters, sort key, cursor, freshness, field visibility, maximum page size.
- **Async work:** return 202 + job resource, webhook, SSE, or notification. Do not hold a request for minutes.

B. API DEFAULTS

- Resource-oriented HTTP for external APIs; gRPC for typed internal RPC; GraphQL when client-driven shape/aggregation truly pays for its complexity.
- Unsafe retries use an **idempotency key** bound to caller + operation + payload hash, with a stored terminal response and retention window.
- **Cursor pagination** over a stable total order, commonly `(created_at, id)`. Encode an opaque cursor; define snapshot vs live semantics.
- Rate limits need scope (user, tenant, IP, token cost), algorithm, burst size, and 429 Retry-After. Bound page size and query cost too.
- Errors: HTTP status + machine code + safe message + request/trace ID. Retry only explicit transient classes.

CORRECTION TO COMMON ADVICE

PUT is defined as idempotent at the protocol-semantic level, but a buggy implementation can still duplicate side effects. Idempotency is a property you enforce and test, not one you inherit from a verb.

C. QUERY-FIRST MODELING

- List the dominant reads and writes first. Then pick the store and keys. A NoSQL data model is often **one table or projection per access pattern**.

NEED	GOOD START	WATCH
Transactions, joins, constraints	Postgres/MySQL	hot rows, connection count
Key-based scale, predictable latency	Dynamo-style KV	rigid queries, hot partitions
Wide write-heavy rows	Cassandra/LSM	compaction, read amplification
Text relevance/facets	Search index	lag; not source of truth
Multi-hop traversal	Graph DB	partitioning, supernodes
Blobs	Object store	metadata/ACLs live elsewhere

D. NORMALIZATION IS A DIAL

- Normalize authoritative mutable state to prevent update anomalies.
- Denormalize read paths when the read/write ratio and latency justify staleness and repair machinery.
- A purchased line item copying price/name is a **historical snapshot and correctness boundary**, not merely a performance hack.
- For every duplicate field: name the owner, propagation mechanism, acceptable lag, and reconciliation job.

E. INDEXES: MATCH THE QUERY

- **B-tree:** equality and range. Composite order usually equality columns, then range/sort; verify the leftmost-prefix behavior.
- **Covering/index-only:** include projected columns for a hot narrow query; pay extra write and storage cost.
- **Partial:** index only the hot subset, such as active rows. **Unique:** make invariants and dedupe race-safe.
- **GIN/inverted:** terms/arrays/JSON. **GiST/R-tree:** spatial/range. **BRIN:** huge physically ordered tables. **HNSW/IVF:** approximate vector search.
- Avoid standalone low-cardinality indexes. Build large indexes with an online/concurrent migration plan and monitor write amplification.

F. IDS, TENANCY, AND LIFECYCLE

- Use time-sortable globally unique IDs (UUIDv7/Snowflake-like) when you need decentralized creation plus index locality; hide internal IDs when enumeration matters.
- Shared multi-tenant tables usually carry `tenant_id` through keys, indexes, authorization, quotas, and shard routing.
- Lifecycle: retention, soft vs hard delete, tombstones, legal hold, backup expiry, compaction/vacuum, and restore testing.
- **Schema evolution: expand → backfill → dual-read/verify → cut over → contract.** Never require an atomic fleet-wide deploy.

05 Scale reads and writes deliberately

A. ESCALATION ORDER

READS	WRITES
Fix query/index/connection use	Fix transaction/query; vertical headroom
CDN and client/browser cache	Batch within one transaction/round trip
Cache-aside hot objects	Queue non-interactive work
Read replicas / projections	Partition by access pattern
Denormalize/precompute	Adopt write-optimized log/LSM

- Move up only when a number, SLO, or operational constraint forces it.

B. CACHE DECISION CARD

- **What:** object, query, page fragment, computation, negative result, or model response?
- **Where:** browser/CDN, process-local, distributed cache, or materialized projection?
- **Freshness:** TTL, explicit invalidation, versioned key, or never-expire + async refresh?
- **Ownership:** source of truth remains elsewhere; define refill and repair.
- **Economics:** expected hit rate, object size, eviction behavior, network hop, and origin capacity on miss.

C. CACHE PATTERNS

- **Cache-aside:** default; app loads on miss. Resilient to cache loss, but first miss is slow and stale invalidation is yours.
- **Read-through:** cache library loads. Cleaner call sites; tighter coupling to the cache layer.
- **Write-through:** update cache in the write path. Fresher reads; higher write latency and dual-write care.
- **Write-behind:** buffer/flush async. Fast writes; durability, ordering, and recovery become hard.

D. CACHE FAILURE MODES

- **Stampede:** request coalescing/single-flight, jittered TTL, probabilistic early refresh, stale-while-revalidate.
- **Hot key:** local cache, replicate the key, request coalescing, split the object, special-case the celebrity path.
- **Cold start:** warm critical keys, ramp traffic, preserve origin headroom, shed optional work.
- **Poisoned/stale data:** schema/version in key, bounded TTL, invalidate on rollback, reconciliation.

- **Cache outage:** bypass only at a safe rate; otherwise a cache failure becomes a database failure.

SAY THIS

"The cache is an optimization, not a correctness dependency. I will define a bounded-staleness contract and an origin protection mode before relying on its steady-state hit rate."

E. READ REPLICAS AND PROJECTIONS

- Replicas scale read QPS and isolate workloads, but introduce lag, stale reads, failover behavior, and connection routing.
- **Read-your-writes options:** pin the author to the primary briefly, carry a write timestamp/LSN token, read from a write-through cache, or wait until the replica catches up.
- A materialized view/search index is a read-optimized copy. Name the freshness SLO, rebuild path, source of truth, and drift detection.

F. PARTITIONING / SHARDING

- Choose the shard key from access patterns and distribution. Goal: common queries hit one shard, and hot entities do not share one partition.
- **Hash:** even point access; poor ranges. **Range:** efficient scans; hot tail. **Directory:** flexible placement; metadata service dependency.
- Consistent hashing reduces movement as nodes change; virtual nodes improve balance. It does **not** solve a single hot key.
- Costs: cross-shard joins/transactions, scatter-gather tails, global uniqueness, resharding, backup/restore, and operational fanout.
- Plan resharding: virtual buckets, dual-write/dual-read, backfill, checksum, cutover, and rollback.

G. WRITE-HOT DATA

- Batch inserts and group commits; keep transactions short.
- Append events and compact/aggregate later when updates are not required in place.
- **Shard counters** into N cells; sum on read or periodically materialize.
- Route a hot entity to a single serialized worker when ordering and contention dominate.
- LSM stores trade high sequential write throughput for compaction and read amplification. Mention only when workload shape justifies it.

06 Go asynchronous without losing control

A. QUEUE OR LOG?

NEED	WORK QUEUE	DURABLE LOG / STREAM
Primary goal	distribute jobs	retain ordered history
Consumption	one worker handles item	many independent consumer groups
Replay	usually limited/DLQ	core capability
Ordering	often best effort / scoped	within partition
Use	email, thumbnails, webhooks	CDC, analytics, projections

B. ACKNOWLEDGMENT SEMANTICS

- **Request accepted** = persisted to a durable queue/log. The client sees pending, not success of the downstream effect.
- **At-most-once**: may lose, avoids duplicates. **At-least-once**: retries, may duplicate. Most business pipelines start here.
- **Exactly-once processing** is possible inside some bounded systems/transactions. Exactly-once **business effect** still requires idempotency, dedupe, or atomic sink semantics end to end.
- Choose the acknowledgment boundary from product truth: payment authorized, order recorded, email queued, or email delivered are different states.

C. IDEMPOTENT CONSUMER

- Stable event ID + semantic operation ID. Dedupe must be scoped and retained longer than the maximum replay window.
- Prefer a database uniqueness constraint or atomic compare-and-set over a read-then-write check.
- If an external side effect supports idempotency keys, propagate one. Otherwise record intent/result and reconcile ambiguous timeouts.
- Handlers should tolerate duplicate, late, and out-of-order events. Version event schemas; never reuse a field with a new meaning.

D. TRANSACTIONAL OUTBOX / CDC

- Write business state and an outbox row in **one local transaction**; a relay publishes to the broker; consumers dedupe.

- CDC can turn database changes into a stream without application dual writes, but raw row changes may be a poor public domain contract.
- An **inbox table** is the consumer-side mirror when processing and dedupe must commit atomically.

DUAL-WRITE TEST

If the process crashes after system A commits but before system B commits, can you repair deterministically? If not, the design is not complete.

E. PARTITIONS AND ORDER

- **Ordering is scoped**. Key events by entity so the broker hashes the same entity to the same partition. Do not create one physical partition per entity.
- Partition count bounds parallelism. A hot entity can still monopolize one partition; isolate or special-case if required.
- Retries can reorder. Use sequence/version checks, a retry topic, or block a key; choose throughput vs strict per-key order.
- Global total order is expensive and usually not a product requirement. Ask **what must be compared**.

F. BACKPRESSURE, RETRIES, AND DLQ

- Bound queue depth, in-flight work, concurrency, payload, retry count, and per-tenant share.
- Exponential backoff + jitter + retry budget. Retry only transient errors. Put retry policy at **one layer** to prevent multiplication.
- DLQ is quarantine, not resolution: alert, inspect, fix, replay safely, and track age/count by reason.
- Autoscale on **queue age and service time**, not only message count. Protect the downstream dependency with concurrency limits.

G. PIPELINE OPERATIONS

- Metrics: ingress/egress rate, oldest age, lag by partition, attempts, poison rate, processing latency, and sink errors.
- Replay plan: source retention, schema compatibility, side-effect suppression/dedupe, rate-limited catch-up, and progress checkpoints.
- Stream windows: event time vs processing time, watermark, allowed lateness, corrections, and state retention.

07 Protect correctness under concurrency

A. START WITH THE INVARIANT

- Examples: inventory never below zero; one booking per slot; ledger entries balance; a message edit must not overwrite a later edit. Then choose the **cheapest mechanism that enforces it at the authoritative boundary**.

B. CONTENTION TOOLBOX

- **Atomic statement:** `UPDATE inventory SET qty=qty-1 WHERE id=? AND qty>0`; check rows affected.
- **Constraint:** unique/exclusion/check/foreign-key turns races into explicit failures.
- **Optimistic concurrency:** compare version/ETag; retry or return conflict. Great at low contention; retry storm at high contention.
- **Pessimistic lock:** lock row/range in a short transaction. Predictable under hot contention; deadlocks and throughput cost.
- **Serialized actor/queue:** one logical executor per key. Adds latency and recovery/ownership complexity.
- **Distributed lock:** last resort for cross-system critical sections. Lease expiry needs a **fencing token checked by the protected resource**.

C. TRANSACTION ISOLATION

LEVEL	WHAT TO REMEMBER
Read committed	each statement sees committed data; read-modify-write still races
Repeatable read / snapshot	stable snapshot; write skew may exist by database semantics
Serializable	equivalent to serial execution; expect abort/retry and lower throughput

- **Name the anomaly you are preventing.** Do not use "strong consistency" as a substitute for a transaction/isolation decision.

D. CROSS-SERVICE WORKFLOWS

- **Saga:** local transactions + durable workflow state + compensating actions. Compensation is a business action, not database rollback.
- **Orchestration:** one workflow owner makes state visible and retries explicit. **Choreography:** less central coupling, but flows are harder to understand and change.
- **2PC:** atomic commit across participants, but availability/operational costs and participant support constrain use. Do not dismiss or select it by slogan.

- Model irreversible steps **late**. Use pending/reserved states and reconciliation for ambiguous outcomes.

E. CONSISTENCY MENU

- **Linearizable:** operations appear in one real-time order. Use for a critical authoritative decision.
- **Sequential/per-key order:** all observers agree on order, not necessarily wall-clock recency.
- **Session:** read-your-writes and monotonic reads for one user/session. Often the right product contract.
- **Bounded staleness:** data no older than a time/version bound. Easier to explain than vague "eventual."
- **Eventual:** replicas converge without new writes. Must still define conflict and user experience.

CAP, USED CORRECTLY

CAP constrains behavior during a network partition: preserve consistency by refusing/limiting operations, or preserve availability by accepting divergent/stale behavior. It is not a three-way database feature score and does not replace latency tradeoffs in normal operation.

F. REPLICATION AND CONFLICTS

- **Single-leader:** simple write order; leader failover and cross-region write latency are the key risks.
- **Multi-leader:** local writes and regional survival; conflicts, duplicate effects, and convergence become application problems.
- **Leaderless/quorum:** tune R/W/N, repair/read reconciliation, hinted handoff; quorums alone do not guarantee linearizability under every failure.
- Conflict policies: reject, last-write-wins (clock risk/data loss), field-level merge, domain merge, or CRDT where operations fit.

G. MONEY AND INVENTORY

- Keep an **append-only ledger**; balances are derived/materialized. Never overwrite financial history.
- Reserve inventory with expiration, then confirm/cancel. Unique/atomic enforcement at the source of truth.
- Separate authorization, capture, refund, and settlement states. Reconcile with the external provider.
- Audit every transition with actor, request ID, operation ID, timestamp, and reason.

08 Realtime delivery, fanout, search, and geo

A. CLIENT DELIVERY LADDER

MECHANISM	USE	COST
Polling	seconds/minutes freshness	waste; simplest/stateless
Long polling	near-realtime on plain HTTP	held requests; reconnect churn
SSE	server → client stream, tokens	one-way; connection limits/proxies
WebSocket	bidirectional low latency	state, heartbeat, resume, routing
Push	background/mobile wake-up	best effort; platform limits

- Start with the simplest mechanism that meets freshness and interaction needs. Realtime delivery does not imply realtime durable storage.

B. CONNECTION PLANE

- Gateway authenticates, rate limits, maintains heartbeat, and tracks subscriptions. Keep application workers stateless where possible.
- Registry maps user/session → gateway with TTL, or gateways subscribe only to channels for locally connected users.
- On reconnect, the client sends its last cursor/sequence; the server replays from a durable log/store, then resumes live delivery.
- Slow consumers need bounded buffers, a coalescing/drop policy, snapshot fallback, or disconnect. **Never allow unbounded per-socket memory.**
- Presence/typing/read receipts are often lossy TTL state. Durable chat messages are not. **Separate the contracts.**

C. FANOUT

- **On write:** precompute recipient inboxes. Fast reads; write/storage amplification and the celebrity problem.
- **On read:** fetch sources and merge. Cheap writes; expensive read and ranking latency.
- **Hybrid:** push normal producers, pull high-fanout producers, merge/rank on read. Quantify the threshold from follower distribution and read ratio.
- Protect against fanout storms with chunked jobs, per-producer quotas, queue backpressure, and delayed lower-priority delivery.

D. SEARCH AS A PROJECTION

- Source of truth → outbox/CDC → indexer → search cluster. Expose indexing lag and rebuild from authority.

- Define lexical vs semantic relevance, typo tolerance, filters/facets, freshness, languages, access-control filtering, and pagination stability.
- Avoid leaking deleted/private data: authorization-aware indexing, tombstones, a delete pipeline, and periodic reconciliation.
- Rank stages: candidate retrieval → filters → feature hydration → lightweight rank → expensive rerank. Cache by query only when permissions allow.

E. GEO / PROXIMITY

- Geohash/S2/H3 cells create a coarse candidate set; search current and neighboring cells, then compute exact distance.
- Cell size trades candidate count against boundary misses. Adaptive/multi-resolution cells help density skew.
- Store location precision and retention according to privacy need; exact location is sensitive data.

F. COLLABORATIVE EDITING

- A central sequencer/room server is simplest when online and region-local.
- Operational transform or CRDTs support concurrent edits/offline merges; both require operation IDs, version vectors/causal context, compaction, and a reconnect protocol.
- Pick from product needs: offline edits, number of collaborators, object size, undo semantics, and conflict visibility.

G. MEDIA UPLOAD / PROCESSING

- Client obtains a short-lived **presigned upload URL**; bytes go directly to object storage.
- A metadata row tracks upload state; an event triggers scan/transcode/thumbnail; publish only safe completed variants.
- Multipart/resumable upload for large files; content hash for integrity/dedupe; CDN for delivery.
- Quotas, MIME sniffing, malware scan, moderation, lifecycle tiering, and orphan cleanup are part of the design.

SAY THIS

"I will separate durable truth from delivery presence. A reconnecting client can always recover from a cursor, while ephemeral typing indicators may be dropped under pressure."

09 Design for failure, overload, and regions

A. FAILURE IS A PATH

- For each critical dependency: timeout, retry eligibility, retry budget, circuit/open behavior, fallback, data consistency, and alert.
- Distinguish partial, slow, correlated, and Byzantine/bad-data failures. Replication mainly helps some crash failures.
- A health check must reflect ability to serve; readiness prevents sending traffic before warmup; liveness should not cause restart loops.

B. TIMEOUTS, RETRIES, HEDGING

- Set the timeout from the caller's **end-to-end deadline**; propagate the remaining deadline downstream. One slow dependency cannot spend the budget repeatedly.
- Retry only idempotent/transient operations. Exponential backoff + jitter + max attempts + a global retry budget.
- Retry at **one layer**. Three retries across three layers can multiply attempts dramatically.
- A hedged request can cut tail latency for safe reads after a delay, but increases load. Use only with capacity and cancellation.

C. OVERLOAD CONTROL

- Bound every queue, pool, batch, request size, query complexity, and per-tenant share.
- Admission control before expensive work; shed low-priority/optional requests quickly with an explicit overload status.
- Backpressure upstream; degrade features; serve stale cache; reduce fanout; sample non-critical writes.
- Bulkheads isolate dependencies/tenants/work classes so one hot path cannot consume all threads/connections.
- Autoscaling is not instantaneous and cannot fix a saturated database or bad dependency. **Preserve headroom.**

CASCADING FAILURE TEST

When one replica fails, remaining replicas take more load; latency rises; callers time out and retry; load rises again. Break the feedback loop with shedding, retry budgets, concurrency limits, and capacity headroom.

D. AVAILABILITY TOPOLOGIES

TOPOLOGY	GOOD FOR	HARD PART
Multi-zone, one region	common HA baseline	regional outage
Active-passive regions	lower complexity/cost	failover, stale replica, drills
Active-active reads	global read latency	cache/data freshness
Active-active writes	global write availability	conflicts, ordering, cost

E. MULTI-REGION DECISION

- Start from RTO/RPO, write latency, residency, and outage scope. **Multi-region is not automatically better.**
- Choose a home region per tenant/entity when local writes matter but conflicts do not. Route via directory; define home-region loss.
- Asynchronous replication: low local latency, nonzero RPO. Synchronous cross-region: stronger durability/order, physics on every write.
- Failover needs fencing the old primary, traffic switch, replica promotion, dependency readiness, cache warmup, and a fallback plan.

F. DISASTER RECOVERY

- **Backups are not DR until restore is tested.** Replication is not backup against corruption/deletion.
- Define backup frequency, immutability, encryption, retention, restore order, credentials/config recovery, and maximum restore duration.
- Run game days. Measure achieved RPO/RTO. Predefine manual decision authority and user communication.
- For derived stores, rebuild from authority; do not pay to replicate every projection unless RTO requires it.

G. GRACEFUL DEGRADATION

- Rank features: must-work, important, optional. Protect write correctness before recommendations, counts, presence, or freshness.
- Examples: cached catalog without personalization; accept upload for later processing; read-only mode; disable expensive search facets.
- Make degraded state observable and reversible. Avoid mode switches that synchronize every node at once; add jitter.

10 Operate, secure, and evolve the system

A. OBSERVABILITY BY USER JOURNEY

- **Metrics:** rate, errors, duration, saturation plus business correctness (orders recorded, duplicate charges, indexing lag).
- **Logs:** structured, sampled, privacy-safe; operation/request/tenant IDs; never secrets or raw sensitive payloads.
- **Traces:** propagate context through RPC and async events; include queue wait separately from processing time.
- **Profiles:** CPU/allocation/blocking when resource cost is unclear.
- Dashboards follow the critical journey and dependencies; alerts fire on user-impacting SLO burn, not every noisy metric.

B. GOLDEN SIGNALS + DOMAIN SIGNALS

SYSTEM	DOMAIN
p50/p95/p99 latency	checkout success / duplicate effect
Error rate by class	feed/search freshness
CPU/memory/connection saturation	payment reconciliation delta
Queue age / replication lag	abuse block / false-positive rate
Cache hit and origin load	model quality / groundedness

C. SECURITY THREAT PASS

- **Identity:** authentication, token/session lifecycle, service identity, key rotation.
- **Authorization:** object and field level on every read/write; tenant isolation; least privilege. **UUIDs do not replace authZ.**
- **Input/resource abuse:** schema validation, injection defense, file scanning, query/page limits, per-principal quota and cost limit.
- **Data:** TLS, encryption at rest, key ownership, secrets manager, sensitive-field access audit.
- **Supply/operations:** dependency provenance, signed artifacts, secure defaults, audit trail, incident revocation.

D. PRIVACY AND COMPLIANCE

- Data classification and minimization: collect only what the product needs; separate sensitive fields and limit joins.

- Retention/deletion must cover primary DB, replicas, caches, indexes, analytics, logs, **embeddings**, and backups according to policy.
- Residency and transfer constraints affect sharding and failover. Consent/purpose can affect downstream uses.
- Audit access without leaking values. Design export/delete as durable workflows with progress and evidence.

E. SAFE ROLLOUT

- Feature flag → internal/dark traffic → small canary → staged percentage/region/tenant → full rollout.
- Define success metrics and automatic rollback **before** launch. Compare canary to control, especially p99 and correctness.
- Backward-compatible protocol and schema first; old and new binaries coexist during rollout.
- Shadow reads/writes can validate a new path, but suppress side effects and budget the extra load.

F. ZERO-DOWNTIME DATA MIGRATION

- **Expand:** add optional field/table/index; deploy writers compatible with old readers.
- **Backfill:** chunk, throttle, checkpoint, make idempotent, observe lag/load.
- **Verify:** dual-read or compare sampled checksums/invariants; reconcile mismatches.
- **Cut over:** switch reads behind a flag; maintain a rollback window.
- **Contract:** stop old writes; remove old schema only after the fleet and rollback window are clear.

G. OWNERSHIP AND OPERABILITY

- Name the service and data owner, on-call boundary, dependency SLO, runbook, and capacity owner.
- Minimize operational fanout: 1,000 tenant databases improve isolation but multiply schema rollout, backup, alerting, and incident work.
- **Build vs buy:** differentiate product-critical logic; use managed primitives when they meet correctness, cost, lock-in, and compliance needs.

STAFF+ CLOSING LINE

"The architecture is not done when steady state works. I need a safe rollout, a repair path for derived data, an overload mode, and an owner who can operate it at 3 a.m."

11 AI-native and LLM application systems

A. MODEL CALL AS AN EXPENSIVE DEPENDENCY

- Budget time and tokens end to end. Stream early output when it improves perceived latency; propagate cancellation to stop wasted work.
- Timeout, bounded retry for transient failures, per-user token/rate quota, concurrency limit, and a fallback/degraded mode.
- Persist request state before long work; use an async job/background mode when the client connection need not stay open.
- **Version model, prompt, tools, retrieval config, safety policy, and response schema** for reproducibility and rollback.

B. CHAT AND CONTEXT

- Conversation is **append-only messages plus mutable metadata**. Define edit/delete and partial-generation semantics.
- Context assembly: system policy + relevant user state + recent turns + summary/retrieval. Enforce a token budget and provenance.
- Do not trust client-provided history for authorization or billing. Store authoritative conversation and tool outcomes server-side.
- The streaming protocol needs event types, sequence, resume cursor, final usage, error, and cancellation; persist final/partial state intentionally.

C. RAG PIPELINE

- **Ingest:** parse → chunk → enrich metadata/ACL → embed → index. Keep the original source and version; processing is restartable.
- **Query:** authorization filter → lexical + vector retrieval → dedupe → rerank → context pack → generate with citations.
- **Evaluate retrieval separately:** recall@k, ranking, freshness, permission leakage. Then evaluate answer groundedness and task success.
- Updates/deletes propagate through durable jobs; reconcile the index against the source. Cache only with tenant/ACL/version in the key.
- Chunk size/overlap and top-k are **workload parameters, not universal constants**.

D. TOOL-USING AGENT LOOP

- Planner/model proposes a typed tool call → policy/authZ validates → executor uses idempotency + timeout → result is recorded → model continues.
- **Treat model output as untrusted input.** Validate JSON/schema, allowed tool, argument bounds, resource ownership, and user confirmation for high-impact actions.
- Bound steps, wall-clock, tokens, spend, fanout, and recursion. Detect repeated/no-progress calls.
- Durable state machine/checkpoints for long jobs; resume after a worker crash **without repeating external effects**.
- Human approval at irreversible or high-risk boundaries; make the pending action and audit trail visible.

E. PROMPT INJECTION AND DATA BOUNDARIES

- Separate trusted instructions from untrusted retrieved/user content; **documents must not grant tool authority**.
- Least-privilege tools, scoped credentials, sandboxing, network/domain restrictions, output encoding, and confirmation gates.
- Prevent cross-tenant retrieval, secret exfiltration, excessive tool parameters, and unsafe content propagation.
- Log decisions and tool metadata safely; redact sensitive prompt/tool payloads according to policy.

F. QUALITY, SAFETY, AND EVALS

- Offline golden set before any prompt/model change; stratify by critical slices and adversarial cases.
- Use deterministic checks where possible, model graders with calibration, and human review for high-impact quality.
- Online: task completion, user correction, groundedness, safety violations, latency, token cost, abandonment, and rollback guardrails.
- Canary model/prompt versions; retain a comparison and replay corpus within privacy policy. **Prompts/config are versioned production artifacts.**

SAY THIS

"The model is probabilistic, but the surrounding system does not have to be. I will make tool authority, state transitions, budgets, and side effects deterministic and auditable."

12 The LLM product blueprint: walk-through, anchors, cost

A. CHATGPT-LIKE REFERENCE WALK-THROUGH

- **Send path:** client POST → gateway (auth, per-user rate/token quota) → chat service persists the user message + an assistant-message placeholder (status: generating) → context builder (system policy + recent turns + summary + ACL-filtered retrieval, packed to token budget) → inference orchestrator (model routing, admission queue, streaming) → provider/model pool.
- **Stream path:** tokens flow back through the gateway to the client via SSE, with sequence numbers; the orchestrator appends chunks to the placeholder (buffered writes, not per token); on completion persist final text + usage + finish reason.
- **Failure path:** client disconnect ≠ generation failure — decide whether generation continues server-side; on reconnect the client sends its message ID + cursor and replays from the persisted partial, then resumes live. Provider timeout → bounded retry or degrade to a smaller model → explicit error state preserving retry context.
- **Async siblings:** title generation, conversation summarization/memory, moderation, and eval sampling hang off the completed-message event — never in the interactive path.
- Source of truth: the conversation store. Everything else — retrieval index, summaries, analytics — is a rebuildable projection (§08.D applies unchanged).

B. LLM ESTIMATION ANCHORS

- **Latency splits in two:** time-to-first-token ≈ queueing + prefill, and prefill grows with prompt length; decode speed is commonly **tens of tokens/second per stream**. A ~500-token answer takes seconds to tens of seconds — which is *why* streaming is non-negotiable, not a nicety.
- **Connections are the scarce resource, not QPS.** Little's Law on streams: $100 \text{ requests/s} \times 10 \text{ s average stream} = \mathbf{1,000 \text{ concurrent open connections}}$. Size gateways by concurrent connections and per-connection memory; long-lived SSE changes load-balancer and proxy behavior.
- **Cost is per token, asymmetric:** output tokens typically cost several times input tokens; long context inflates both

cost and TTFT. Trim, summarize, and cap output length before touching model choice.

- **Prefix/prompt caching:** a stable system prompt + tools prefix can be cached by the provider — order context so the stable part comes first.
- **Vector math:** a ~1,500-dim float32 embedding ≈ 6 KB. 10M chunks ≈ 60 GB of raw vectors plus index overhead — memory-bound, so quantization, IVF/disk indexes, or tiering enter the design. Docs commonly split into tens of chunks of a few hundred tokens each.

C. COST AND LATENCY LEVERS

- Reduce round trips first; parallelize independent work; stream; trim output; compact context; batch offline workloads.
- **Route by difficulty/risk to a model tier;** cache stable prefixes or safe deterministic results; precompute embeddings.
- Speculative/parallel tool or model calls trade cost for latency. Cancel losers and cap fanout.
- Track **cost per successful user outcome**, not only per request. A cheap failed answer is expensive.

D. DEEP-DIVE PROMPTS TO EXPECT (PEOPLE-INNOVATION FLAVOR)

- "Internal knowledge assistant": press on ACL-aware retrieval (permission leakage is the kill shot), ingestion from changing sources, freshness SLO, and eval design for correctness on internal facts.
- "Recruiting/HR copilot": press on sensitive-data classes in prompts/logs/embeddings, human approval before external effects (emails, offers), and auditability of every model-influenced decision.
- "Chat with org data": press on the hybrid lexical+vector retrieval decision, context budget under long documents, citation faithfulness, and cost per seat.

SAY THIS

"Two numbers shape this design: streams live for seconds, so I size the connection plane by concurrency, not QPS; and tokens are the cost unit, so context assembly is a budgeted, ordered, cacheable pipeline — not string concatenation."

13 Decision matrices, prompt map, and final drill

A. STORE / TRANSPORT QUICK CHOOSER

IF THE DOMINANT NEED IS...	START WITH...	FORCE TO ADD
Transactional product state	relational DB	measured key/write bottleneck
Key-addressed massive scale	managed KV / wide-column	rigid access pattern accepted
Hot ephemeral acceleration	Redis/cache	bounded staleness + origin plan
Text/semantic retrieval	search/vector projection	rebuild + ACL + freshness plan
One job handled once-ish	work queue	retry/idempotency/DLQ
Replay to many consumers	durable log	partition/order/retention plan
Large immutable bytes	object store + CDN	metadata, ACL, lifecycle

B. PROMPT → LIKELY DEEP DIVE

PROMPT	PRESS ON
Feed / timeline	fanout hybrid, ranking, hot creators, pagination
Chat / notifications	realtime resume, ordering, presence, push
Ticketing / inventory	atomic reserve, contention, expiry, fairness
Payments / ledger	idempotency, state machine, reconciliation, audit
File/photo/video	direct upload, async processing, CDN, moderation
Search / maps	index freshness, ranking, geo cells, ACL filtering
Metrics / trending	stream windows, lateness, rollups, hot keys
Collaboration	sequencing, OT/CRDT, offline/reconnect, compaction
LLM assistant	streaming, context, tools, RAG, evals, budgets (\$11-12)

C. THE FIVE-MINUTE CLOSING CHECK

- **Scope:** Did I satisfy the stated user journeys and cut nonessential features explicitly?
- **Numbers:** Did scale, latency, SLO, RPO/RTO, and geography force my major choices?

- **Correctness:** Source of truth, invariant, duplicate/order/staleness semantics, reconciliation?
- **Failure:** Timeouts, retry budget, overload/degraded mode, region/data recovery?
- **Evolution:** Observability, security/privacy, rollout, migration seam, ownership and cost?

D. TRADEOFF LINES WORTH REHEARSING

- "I am choosing the simple baseline and naming the measurable trigger for replacing it."
- "That is a product consistency decision: what exactly may this user see stale, and for how long?"
- "This improves steady state but adds a failure mode. Here is the blast radius and degraded behavior."
- "We can pay at write time or read time. The read ratio and freshness target decide."
- "Before sharding, I would validate query shape, one-node headroom, caching, and replicas with a benchmark."
- "I would launch behind a flag, backfill idempotently, compare results, cut over, and keep rollback."

FINAL STAFF+ TEST

Can you explain not only why the system scales, but why the organization can safely launch, operate, repair, migrate, and simplify it?

E. PRIMARY REFERENCES USED FOR THIS EDITION

- **Google SRE Book:** SLOs; Handling Overload; Addressing Cascading Failures — sre.google/sre-book/
- **Amazon Builders' Library:** Timeouts/Retries with Jitter; Idempotent APIs; Load Shedding — aws.amazon.com/builders-library/
- **PostgreSQL docs:** indexes, partitioning, locking, isolation — postgresql.org/docs/current/
- **OpenTelemetry:** signals, context propagation — opentelemetry.io/docs/concepts/
- **OWASP API Security Top 10 (2023):** object-level authZ, resource consumption — owasp.org/API-Security/
- **Google Cloud Architecture Center:** DR planning, RPO/RTO, multi-region — cloud.google.com/architecture/
- **OpenAI platform docs:** streaming, background mode, prompt caching, evals, latency/cost — platform.openai.com/docs/